

NexaCRM

نظام إدارة علاقات العملاء (CRM) لمتجر إلكتروني — توثيق شامل للمشروع والمناقشة

Frontend (React) + Backend (Node.js) + Database (SQLite)

1. فكرة المشروع

NexaCRM هو نظام ويب متكامل لإدارة علاقات العملاء في متجر إلكتروني يبيع منتجات تكنولوجية (لابتوبات، موبايلات، صوتيات، شاشات، إكسسوارات...). الفكرة إن صاحب المتجر يلاقي في مكان واحد: **لوحة متابعة (Dashboard)** بأهم الأرقام، وإدارة **المنتجات** (مخزون وأسعار)، وإدارة **العملاء** (بيانات وقيمة كل عميل)، وإدارة **الأوردرز** (حالات وتتبع)، و**صفحة تحليلات وتقارير (Analytics)** بتشرح أداء المتجر بالرسوم البيانية — وكل ده شغال على **باك إند وداتابيز حقيقية** مش بيانات ثابتة في الكود.

2. المشروع بيخدم مين ويبحل إيه؟

- **صاحب / مدير المتجر:** يشوف الإيراد والأوردرز النشطة والمنتجات اللي مخزونها قليل في لحظتها، وياخد قرارات (إعادة طلب مخزون، عروض للعملاء المهمين).
- **فريق المبيعات / خدمة العملاء:** يلاقي بيانات أي عميل وأوردراته وحالته بسرعة، ويضيف أو يعدّل من واجهة واحدة.
- **المشكلة اللي بيحلها:** بدل الشغل اليدوي أو ملفات متفرقة، كل البيانات في داتابيز واحدة، وكل الأرقام في كل الصفحات **متسقة ومحدثة لحظيًا** (أي أوردر جديد بيسمّع في الداشبورд والتحليلات وقيمة العميل فورًا).

3. هيكل المشروع

المحتوى	المجلد
تطبيق React (الصفحات والمكونات والتنسيق) — بيتكلم مع الباك عن طريق REST API	frontend/
سيرفر Express + Node.js + داتابيز SQLite (ملف <code>crm.db</code> بيتولد أوتوماتيك)	backend/

4. اللي اتعمل خطوة بخطوة

الخطوة 1 — توحيد التنسيق على Tailwind: المشروع كان مخطط بين Tailwind و Bootstrap (صفحة Orders كانت Bootstrap عبر CDN). شلنا Bootstrap نهائياً من index.html وحولنا صفحة Orders ل Tailwind بنفس الـ ديزاين ولغة باقي الصفحات (نفس الهيدر والفلاتر والجدول والمودال).

الخطوة 2 — بناء الباك إند: فولدر backend/ بـ Node.js + Express + SQLite (مكتبة node:sqlite المدمجة، من غير سيرفر داتايز خارجي). ملف db.js بينشئ 3 جداول (products / customers / orders) ويزرع بيانات افتتاحية: 16 منتجاً تكنولوجياً، 10 عملاء، 8 أوردرز.

الخطوة 3 — REST APIs كاملة: عمليات CRUD لكل كيان (عرض/إضافة/تعديل/حذف) + بحث وفلترة + /api/stats و /api/dashboard و /api/analytics + نظام auth (تسجيل/دخول/جلسات بتوكن + تشفير باسورد bcrypt). أي IDs فيها # بتتبع مشفرة (encodeURIComponent) عشان متتكسرش في الـ URL.

الخطوة 4 — ربط الفرونت وشيل الداتا الثابتة: ملف مركزي src/lib/api.js بقى المصدر الوحيد للداتا. الصفحات الثلاثة (Products / Customers / Orders) بقت تجيب وتعرض وتعديل عبر الـ API، واتشال ملف الداتا الثابت وحفظ الـ localStorage، مع حالات تحميل ورسائل خطأ لو الباك مش شغال.

الخطوة 5 — ربط الداشبور: صفحة Dashboard كانت أرقام وهمية ثابتة، بقت كلها من /api/dashboard (الإيراد، الأوردرز النشطة، العملاء، المنتجات، آخر الأوردرز، الأعلى مبيعاً، ترند الإيراد). وزار **View all** بقى يؤدي لصفحة Orders.

الخطوة 6 — اتساق الأرقام: اتأكدنا إن أرقام الداشبور = نفس داتا باقي الصفحات (نفس الإيراد، نفس العدد). وبقي **LTV وعداد أوردرز كل عميل محسوبين أوتوماتيك** من جدول الأوردرز بدل القيم المخزنة، ووحدنا العملة على \$ في كل الصفحات، وحطينا dropdown للعملاء في مودال إضافة الأوردرز عشان الربط ميتكسرش.

الخطوة 7 — دمج شغل الفريق (Auth): زميلنا ضاف نظام دخول (جداول users/sessions + صفحتي SignIn/SignUp + حماية الصفحات). حصل conflicts كتير مع شغلنا فحليناها كلها (أخذنا نسخته لأنها كانت شاملة شغلنا + الزيادة، ووَلَدنا الـ lockfile من جديد، وشلنا ملفات الداتايز اللي اترفعت بالغلط) ودمجنا في main بعد تجربة كاملة.

الخطوة 8 — صفحة Analytics بتصميم Figma: نفذنا صفحة Analytics & Reports بنفس شكل التصميم (هيدر + Export CSV/PDF شغالين + كروت KPIs + أعمدة الإيراد الشهري + خط الأوردرز والإيراد لآخر 7 أيام + دونات قنوات البيع + أعمدة الإيراد حسب الفئة). كارت الـ Traffic اتبدل بما يكافئه من الداتا الحقيقية (مفيش داتا زيارات في المشروع)، واتضاف حقل قناة البيع للأوردر عشان الدونات تبقى حية.

الخطوة 9 — الهوية: اسم المشروع NexaCRM Dashboard وأيقونة بنفسجية بسهم نمو مطابقة للوجو.

5. التقنيات المستخدمة

الجزء	التقنية	ليه؟
واجهات	React 19 + React Router 7 + Vite	سرعة وبناء صفحات تفاعلية وتنقل داخلي
تنسيق	Tailwind CSS 4	تصميم موحد وسريع بدون ملفات CSS مشتتة
رسوم بيانية	Recharts	شارتات تفاعلية للداشبورد والتحليلات
أيقونات	Lucide + Font Awesome	أيقونات خفيفة ومتسقة
سيرفر	Node.js + Express 4	API خفيف وسريع بنفس لغة الفرونت
داتابيز	SQLite (مدمجة)	ملف واحد، مش محتاجة سيرفر، مناسبة لمشروع جامعي
أمان	bcryptjs + توكن جلسات	تشفير باسورد وجلسات دخول
جودة	Oxlint + Build Vite	كشف الأخطاء قبل التسليم
إدارة نسخ	Git + GitHub (فروع + PRs)	شغل جماعي منظم وحل تعارضات

6. أهم الـ APIs

المسار	الوظيفة
GET /api/products • /customers • /orders	عرض + بحث وفلترة
POST / PUT / DELETE .../:id	إضافة وتعديل وحذف لأي كيان

تغيير حالة الأوردر	PATCH /api/orders/:id
أرقام مجمعة للصفحات	GET /api/dashboard • /api/analytics • /api/stats
التسجيل والدخول والجلسات	POST /api/auth/register • /login • /logout + GET /me

7. قاعدة البيانات

أهم الحقول	الجدول
category, price, stock, image, (يونيك), name, sku	products
segment, location, orders_count, ltv, joined, (يونيك), name, email	customers
product, price, status, channel, created_at, (بالاسم), id, customer	orders
بيانات الدخول المشفرة + توكنات الجلسات	users / sessions

ملحوظة تصميمية: الربط بين الأوردرز والعملاء/المنتجات **بالاسم** (مش مفاتيح خارجية) — ببسّهل العرض، وال LTV بيتحسب لحظيًا بجمع أسعار الأوردرز غير الملغية لكل عميل.

8. التشغيل

- الباك: `cd backend` ثم `npm install` (أول مرة) ثم `npm run dev` ← يعمل على `http://localhost:5000`
- الفرونت: `cd frontend` ثم `npm install` ثم `npm run dev` ← يفتح على `http://localhost:5173`
- الدخول التجريبي: `demo@nexacrm.com / demo1234`

9. عشان المناقشة — نقاط قوة وأسئلة متوقعة

ليه SQLite مش MySQL؟ مشروع جامعي بفريق صغير — ملف واحد بدون سيرفر، وال SQL واحد، والترقية لأي داتابيز تانية لا تحتاج تغيير المنطق.

إزاي ضمنتوا اتساق الأرقام؟ مصدر واحد للداتا (الداتابيز) + endpoints مجمعة + LTV محسوب لحظيًا بدل التخزين — وتأكدنا بمقارنة فعلية بين الصفحات.

إزاي اتعاملتوا مع ال **conflicts**؟ دمج لوكال مع rename-detection، مراجعة سطر بسطر، توليد ال lockfile من جديد بدل التعديل اليدوي، وتجربة (auth + dashboard) قبل ال push.

الأمان؟ الباسورد hash بـ bcrypt، الدخول بتوكن جلسات، والصفحات محمية بـ ProtectedRoute.

تطوير مستقبلي؟ مفاتيح خارجية حقيقية، صلاحيات أدوار، تنبيهات المخزون، ودفع حقيقي.